Tutorial 09

▼ Overview

Tutorial (More More More Graphs)

- Hash Tables
- Hash Functions

Lab (From Page)

- To explore the implementation of a simple hash table (In Tutorial)
- To gain experience with using hash tables to implement efficient solutions to programming problems (In Tutorial)

▼ Hash Functions

```
// A Potential Hash Function
int hash(char *key, int N) {
    int h = 0;
    char *c;
    for (c = key; *c != '\0'; c++) {
        h = h + *c;
    }
    return h % N;
}
```

▼ How does this function convert strings to ints?

It adds each characters number, ASCII value, to the int.

▼ What are the deficiencies with this function and how can it be improved?

Anagrams e.g. cat and act, listen and silent, have the same value. One could consider position as well to weight via that too.

```
// A Potential Hash Function
int hash(char *key, int N) {
    int h = 0;
    char *c;
    for (c = key, i = 0; *c != '\0'; c++, i++) {
        h = h + (*c * i);
    }
    return h % N;
}
```

Of course any one letter word would now have the same value so maybe start at $i = 1$.

▼ Separate Chaining

▼ What is Separate Chaining For Collisions

When you have a collision you add the element to linked list for that specific array index.

- An array has the item hash function value e.g. the elements of mod 7 would have keys 0, 7, 14 in slot 0 but they would collide so
- At each index of the array we create a linked list of all items.

▼ With Hash table Size N using separate chaining for collisions. Chains are sorted ascending order by key. What are the best and worst case cost for

inserting $k = 2N$ items. Assume insertion cost is measured in terms of number of key comparisons.

- The worst case is O(k^2)
  - Each element is places in the same value on the table and hence leads to inserting elements into a linked list at the end so 1 to start then 2 then 3, up to k which leads to k (k-1) / 2 hence and O(k^2).
- The best case is all items are spread so the worst insert of all would be 1 comparison but some would need 0 comparisons just find the array index. This causes k/2 comparisons so O(k). It should be based on $\lceil k/N \rceil$

▼ What will the average search cost be after the insertions best and worst case?

- For the worst case you are scanning one chain so the average would be k/2 search cost O(k).
- For the best case you will have to search a chain of length two, which means half the time one comparison and the other half 2 comparisons. Hence 1.5 average search cost O(1).

▼ Hash Table Insertion

▼ Hash Function $h(x) = x \% 11$, table 11 entries. Start with empty and insert the below elements.

`11 16 27 35 22 20 15 24 29 19 13`

Key	11	16	27	35	22	20	1
h(x)							

▼ What are the Hash Values of Each Element (Pass)

Key	11	16	27	35	22	20
h(x)	0	5	5	2	0	9

▼ Using separate chaining, chains in ascending order (Pass)

▼ Insert 11, h(x) = 0

0	1	2	3	4	5	6
11						

▼ Insert 16, h(x) = 5

0	1	2	3	4	5	6
11					16	

▼ Insert 27, h(x) = 5

0	1	2	3	4	5	6
11					16	
					27	

▼ Insert 35, h(x) = 2

0	1	2	3	4	5	6
11		35			16	
					27	

▼ Insert 22, h(x) = 0

0	1	2	3	4	5	6
11		35			16	
22					27	

▼ Insert 20, h(x) = 9

0	1	2	3	4	5	6
11		35			16	
22					27	

▼ Insert 15, h(x) = 4

0	1	2	3	4	5	6
11		35		15	16	
22					27	

▼ Insert 24, h(x) = 2

0	1	2	3	4	5	6
11		24		15	16	
22		35			27	

▼ Insert 29, h(x) = 7

0	1	2	3	4	5	6
11		24		15	16	
22		35			27	

▼ Insert 19, h(x) = 8

0	1	2	3	4	5	6
11		24		15	16	
22		35			27	

▼ Insert 13, h(x) = 2

0	1	2	3	4	5	6
11		13		15	16	
22		24			27	
		35				

▼ What is linear probing?

- When a collision occurs, simply go through all remaining slots until there is a spot to put this item.
- If the array is just below full expand it's size, you will need to use one of the below methods if expanding the array as you could be breaking chains that go past the end back to the start of the array, so ensure you are expanding just before full so not everything is joint via a prob.
- This provides the additional problem that now when you remove elements you will break probe paths that assumed that element existed
  - Backshift - A solution to move everything back until you reach an empty cell
  - Tombstone - A solution just add a blank element in that spot of the array which is empty during insertion but occupied during lookup

There are problems with both of these methods outlined on the lecture slides

▼ Using linear probing insert (Pass)

▼ Insert 11, h(x) = 0

0	1	2	3	4	5	6
11						

▼ Insert 16, h(x) = 5

0	1	2	3	4	5	6
11					16	

▼ Insert 27, h(x) = 5

0	1	2	3	4	5	6
11					16	27

▼ Insert 35, h(x) = 2

0	1	2	3	4	5	6
11		35			16	27

▼ Insert 22, h(x) = 0

0	1	2	3	4	5	6
11	22	35			16	27

▼ Insert 20, h(x) = 9

0	1	2	3	4	5	6
11	22	35			16	27

▼ Insert 15, h(x) = 4

0	1	2	3	4	5	6
11	22	35		15	16	27

▼ Insert 24, h(x) = 2

To find an element at 0, especially if it didn't exist you would have to search almost all elements in the table

0	1	2	3	4	5	6
11	22	35	24	15	16	27

▼ Insert 29, h(x) = 7

0	1	2	3	4	5	6
11	22	35	24	15	16	27

▼ Insert 19, h(x) = 8

0	1	2	3	4	5	6
11	22	35	24	15	16	27

▼ Insert 13, h(x) = 2

At this point we would have wanted to expand the array, but we are only inserting these elements, so it is ok. We always want to expand arrays usually the rule is when there are over half the elements compared to the array size.

0	1	2	3	4	5	6
11	22	35	24	15	16	27

▼ What is double hashing?

The same as linear programming except when hitting a collision rather than just moving up indexes in the array, move up by another amount based on a second hash function.

- We will go over the restraints of a double hashing function in the next question.
- We also have to add to the hash function as we want to move, and if we get 0 we are not moving, this is usually just 1.

▼ What is the second hash functions values for each key $h_2(x) = (x \% 3) + 1$?

Key	11	16	27	35	22	20
h_2(x)						

▼ Answer

Key	11	16	27	35	22	20
h_2(x)	3	2	1	3	2	3

▼ Using double hashing with $h_2(x) = (x \% 3) + 1$ (Pass)

▼ Insert 11, h(x) = 0, h_2(x) = 3

0	1	2	3	4	5	6
11						

▼ Insert 16, h(x) = 5, h_2(x) = 2

0	1	2	3	4	5	6
11					16	

▼ Insert 27, h(x) = 5, h_2(x) = 1

0	1	2	3	4	5	6
11					16	27

▼ Insert 35, h(x) = 2, h_2(x) = 3

0	1	2	3	4	5	6
11		35			16	27

▼ Insert 22, h(x) = 0, h_2(x) = 2

0	1	2	3	4	5	6
11		35		22	16	27

▼ Insert 20, h(x) = 9, h_2(x) = 3

0	1	2	3	4	5	6
11		35		22	16	27

▼ Insert 15, h(x) = 4, h_2(x) = 1

0	1	2	3	4	5	6
11		35		22	16	27

▼ Insert 24, h(x) = 2, h_2(x) = 1

To see an element at key 2, with hashing function 1 that doesn't exist would go through majority of the elements, but this would have to be more specific of an element than linear probing

0	1	2	3	4	5	6
11		35	24	22	16	27

▼ Insert 29, h(x) = 7, h_2(x) = 3

0	1	2	3	4	5	6
11		35	24	22	16	27

▼ Insert 19, h(x) = 8, h_2(x) = 2

0	1	2	3	4	5	6

11		35	24	22	16	27

▼ Insert 13, h(x) = 2, h_2(x) = 2

At this point we would have wanted to expand the array, but we are only inserting these elements, so it is ok.

0	1	2	3	4	5	6
11	13	35	24	22	16	27

▼ Double Hashing Restrictions

▼ Insert 10 into the following array with $h(x) = x \% 10$ and $h_2(x) = x \% 3 + 1$

Index	0	1	2	3	4	5	6
Key	0		12		24		3

▼ Answer

h(10) = 0

h_2(10) = 2

- So we just go index 0, 2, 4, 6, 8, ...
- This is because the increment is not relatively prime to the size of the array, 10/2 = 5, so both have a factor of 2. And not only 1 like relative primes should.
- How to ensure this, just make the size prime and any hash function only return numbers less than the size.
- E.g. $h_2(x) = x \% 10 + 1$ with size 11.]

▼ Hash Tables or Balanced Search Trees?

▼ What are the advantages and disadvantages of both? (Answer Inside)

	Advantages	Disadvantages
Hash Tables	Insertion, search and deletion are average O(1) - Very Fast	Worst case complexity of O(n) - Can avoid with good hash functions Needs to be resized Can waste memory if very few items inserted (most of the table goes unused)
Balanced ST	Items are ordered, so can find the largest and smallest element efficiently. Allocates memory as needed, but requires more memory per element (pointers)	Slower that hash table (but still efficient) O(logn) worst case insertion and deletion

▼ Three Sum Problem

▼ Given an array of integers can you find three elements that add to the target S, try to solve in O(n^2)

Hash Table Functions

```
HashTable HashTableNew(void);
void HashTableFree(HashTable ht);
void HashTableInsert(HashTable ht, int key, int value);
bool HashTableContains(HashTable ht, int key);
int  HashTableGet(HashTable ht, int key);
```

```
void HashTableDelete(HashTable ht, int key);
int  HashTableSize(HashTable ht);
```

```
bool threeSum(int arr[], int size, int sum);
```

▼ Hint 1

We want to insert something into a Hash Table but what, we basically want for each element we get to be able to check that all elements previously inserted could give me the a three sum

- So why not add the sum of the current element with every element up to the current position, that means we are adding all previous combinations, so once we reach an element such that a three sum exists, we just check if the hash table contains sum - arr[i]

▼ Answer

Clearly O(n^2) average as HashTableContains and HashTableInsert are O(1) operations

```
bool threeSum(int arr[], int size, int sum) {
    HashTable ht = HashTableNew();

    for (i = 0; i < size; i++) {
        if (HashTableContains(ht, sum - arr[i]) {
            HashTableFree(ht);
            return true;
        }

        for (int j = 0; j < i; j++) {
            HashTableInsert(ht, arr[i] + arr[j], 0);
        }
    }
```

▼ Find a Secondary Hash Function for size 2521 array (Prime)

▼ Answer

This will mean a slot will not be revisited until all other slots have been visited.

h_2(k) = k % 2520 + 1

▼ Memoisation

Calls are often expensive to functions think recursion, you call the very bottom step a lot of times potentially.

We often use a trick called memoisation and cache the data from the first call. Consulting the cache for future calls.

▼ Design an ADT for a memoiser that supports memoisation of an expensive single arg function

Standard creation and destruction operations

call(x) operation to call the memoised function with x

Use a hash table to store key value pairs, first look at the Hash Table, then compute f(x) and return after putting it into the hash function

▼ A Problem with Memoisation is Memory usage (often we limit the size of the cache with a policy). What is an example policy?

- Evict the oldest
- Evict the least recently used
- Evict the least frequently used

▼ How can you implement these policies?

The first

- Keep a queue of key value pairs an remove the pair dequeued

The second

- Keep a running count internally
- Values act like a timestamp of the call, search for the smallest when doing an eviction.

The third

- Use a separate data structure to store the amount of calls, search for the input with the least calls.

Challenge

- There are quicker options for 2 and 3 but these are harder to implement